Algorithm Lab

Week 3: Binary Search

Description

Binary search in a well-known divide and conquer algorithm. In original, binary search is designed for searching position of specific value in a sorted random-accessible list.
We can see an array $A = \{a_1, a_2, \ldots, a_n\}$ as a function $A'(i) = a_i$ have domain $[1, n]$. If array A is an increasing-order array or a decreasing-order array, then the correspond function $A'(i)$ is a monotone function. Thus, we can change the definition as followed:

Instance: A monotone function $f(X) \to V$, a value $v \in V$, and a precision ε.

Result: $x \in X$ that v in range $[f(x-\varepsilon), f(x+\varepsilon)]$.

In this version, we can not only find the specific value in a sorted array (use $\varepsilon = 1$), but also solve an equation $y = f(x)$ where y is the specific value and $f(x)$ is a monotone function.

Questions

1 Write pseudo code of binary search algorithm.
2 Analysis pseudo code of step 1.
  2.1 Space complexity
  2.2 Time complexity in base case
  2.3 Time complexity in worst case
3 Implement your algorithm to solve ALG02A on https://oj.csie.ndhu.edu.tw/

Space complexity = O(1)#

Time complexity : $\log\left(\frac{R}{A}\right)$

in search range R , we need to search our target in accuracy A

```
function calculate(mid: float, pie: array, F: integer) -> boolean: //確認切片數是否>人數
    total_slices = 0 //總切片數
    for slice in pie:
        total_slices = total_slices + floor(slice / mid) //獲得所有派都切mid大小時能得幾塊
    if total_slices >= F: //若大於人數
        return true
    else:
        return false

function main():
    D = 資料個數
    for j = 0 to D - 1:
        N =派數量
        F = 人數
        F = F + 1
        sum = 0
        pie = []

        //計算所有派的大小
        for i = 0 to N - 1:
            R = read_input()
            pie.push(R * R * pi) //將派面積加入陣列
            sum = sum + pie[i] //加總所有pie面積

        left = 0
        right = sum
        while right - left >目標精度:
            mid = (right + left) / 2
            if calculate(mid, pie, F):  //切片數>人數
                left = mid  //更新左界
            else:
                right = mid //更新右界
        output mid
```

```cpp
const double pi=acos(-1);
vector<float> pie;

bool cal(double mid,int N,int F)
{
    double num=0;
    for (int i=0;i<N;i++)
        num+=floor(pie[i]/mid);
    if (num>=F) return true;
    return false;
}

int main()
{
    int D,N,F;
    cin >> D;

    for(int j=0;j<D;j++)
    {
        cin >> N >> F;
        F+=1;
        double sum=0;
        pie.clear();
        for (int i=0;i<N;i++)
        {
            float R;
            cin >> R;
            pie.push_back(R*R*pi);
            sum+=pie[i];
        }

        double left=0,right=sum;

        while (right-left>1e-5){
            double mid=(right+left)/2;
            if (cal(mid,N,F)) {left=mid;}
            else              {right=mid;}
        }

        cout << fixed << setprecision(4) << left << endl;
    }
    return 0;
}
```